Abstract Syntax and Parsing

CS 350

Dr. Joseph Eremondi

February 12, 2025

Outline

The Big Picture

Describing Syntax

Parsing and Abstract Syntax

The Big Picture

Life of a program

- The Language Pipeline:

Source code	parsing ⟶	Abstract syntax tree	*translation* ⟶	Core Syntax	*evaluation* ⟶	Result
text file	lexing / tokenizing first	data structure	desugaring / compilation	simpler AST / machine code	interpreter, execute on CPU	value, side effects

Syntax vs Semantics

- Can have many syntaxes that parse to the same abstract syntax

Syntax vs Semantics

- Can have many syntaxes that parse to the same abstract syntax
  - Different keywords

Syntax vs Semantics

- Can have many syntaxes that parse to the same abstract syntax
  - Different keywords
  - Different operator names

Syntax vs Semantics

- Can have many syntaxes that parse to the same abstract syntax
  - Different keywords
  - Different operator names
  - Different order of expressions

Syntax vs Semantics

- Can have many syntaxes that parse to the same abstract syntax
  - Different keywords
  - Different operator names
  - Different order of expressions
- E.g. flit/plait vs shplait

Describing Syntax

EBNF

- Extended Backus-Naur form

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol
- Notation for Context Free Grammars

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol
- Notation for Context Free Grammars
  - See CS 411

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol
- Notation for Context Free Grammars
  - See CS 411
- Describes which strings are valid expressions/statements/etc. in a language

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol
- Notation for Context Free Grammars
  - See CS 411
- Describes which strings are valid expressions/statements/etc. in a language
- Generative

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol
- Notation for Context Free Grammars
  - See CS 411
- Describes which strings are valid expressions/statements/etc. in a language
- Generative
  - Gives a process for generating valid strings in the language

EBNF

- Extended Backus-Naur form
  - Named after scientists who worked on Algol
- Notation for Context Free Grammars
  - See CS 411
- Describes which strings are valid expressions/statements/etc. in a language
- Generative
  - Gives a process for generating valid strings in the language
  - String is valid if and only if it’s generated by the grammar

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*
  - A symbolic variable that doesn't show up in the final string, but is replaced using a rule

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*
  - A symbolic variable that doesn't show up in the final string, but is replaced using a rule
- `::=` means *can be one of*

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*
  - A symbolic variable that doesn't show up in the final string, but is replaced using a rule
- `::=` means *can be one of*
- `|` separates the possibilities

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*
  - A symbolic variable that doesn't show up in the final string, but is replaced using a rule
- `::=` means *can be one of*
- `|` separates the possibilities
- Literal strings are in quotation marks

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*
  - A symbolic variable that doesn't show up in the final string, but is replaced using a rule
- `::=` means *can be one of*
- `|` separates the possibilities
- Literal strings are in quotation marks
  - Usually for keywords, operators or parentheses

Example

```
<expr> ::=
    "{" "+" <expr> <expr> "}"
  | "{" "*" <expr> <expr> "}"
  | number
```

- `<expr>` is a *nonterminal*
  - A symbolic variable that doesn't show up in the final string, but is replaced using a rule
- `::=` means *can be one of*
- `|` separates the possibilities
- Literal strings are in quotation marks
  - Usually for keywords, operators or parentheses
- `number` is a literal number e.g. some sequence of digits

Generating a string

- Start with a single non-terminal

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`
  - `<expr>`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`
  - `<expr>`
    - `-> {* <expr> <expr>}`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`
  - `<expr>`
    - `-> {* <expr> <expr>}`
    - `-> {* {+ <expr> <expr>} <expr>}`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`
  - `<expr>`
    - `-> {* <expr> <expr>}`
    - `-> {* {+ <expr> <expr>} <expr>}`
    - `-> {* {+ 5 <expr>} <expr>}`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`
  - `<expr>`
    - `-> {* <expr> <expr>}`
    - `-> {* {+ <expr> <expr>} <expr>}`
    - `-> {* {+ 5 <expr>} <expr>}`
    - `->{ * {+ 5 100000} <expr>}`

Generating a string

- Start with a single non-terminal
  - e.g. `<expr>` for an expression
- Until you have a string with no non-terminals, repeatedly:
  - Replace a non-terminal with one of its variants
    - i.e. one of the things on the right of `::=`
- Examples:
  - `<expr> -> 3`
  - `<expr> -> {+ <expr> <expr>} -> {+ 2 <expr>} -> {+ 2 5}`
  - `<expr>`
    - `-> {* <expr> <expr>}`
    - `-> {* {+ <expr> <expr>} <expr>}`
    - `-> {* {+ 5 <expr>} <expr>}`
    - `->{ * {+ 5 100000} <expr>}`
    - `-> { * {+ 5 100000} -3}`

Parsing and Abstract Syntax

This Looks Familiar

- An expression was:

This Looks Familiar

- An expression was:
  - A number

This Looks Familiar

- An expression was:
  - A number
  - OR, “+” with (an expression AND and expression)

This Looks Familiar

- An expression was:
  - A number
  - OR, “+” with (an expression AND and expression)
  - OR, “*” with (an expression AND an expression)

This Looks Familiar

- An expression was:
  - A number
  - OR, “+” with (an expression AND and expression)
  - OR, “*” with (an expression AND an expression)
- What tools do we have to model something like this?

Parse Trees

- Notice that the different replacements didn’t affect each other

Parse Trees

- Notice that the different replacements didn’t affect each other
    - Can effectively replace them in parallel

Parse Trees

- Notice that the different replacements didn’t affect each other
  - Can effectively replace them in parallel
  - This is what “context free” means

Parse Trees

- Notice that the different replacements didn’t affect each other
  - Can effectively replace them in parallel
  - This is what “context free” means
- Tree structure

Parse Trees

- Notice that the different replacements didn’t affect each other
  - Can effectively replace them in parallel
  - This is what “context free” means
- Tree structure
  - Non-terminal is a node

Parse Trees

- Notice that the different replacements didn’t affect each other
  - Can effectively replace them in parallel
  - This is what “context free” means
- Tree structure
  - Non-terminal is a node
  - Terminal is a leaf

Parse Trees

- Notice that the different replacements didn't affect each other
  - Can effectively replace them in parallel
  - This is what "context free" means
- Tree structure
  - Non-terminal is a node
  - Terminal is a leaf
  - Edge is application of rule from grammar

Parse Trees

- Notice that the different replacements didn't affect each other
  - Can effectively replace them in parallel
  - This is what "context free" means
- Tree structure
  - Non-terminal is a node
  - Terminal is a leaf
  - Edge is application of rule from grammar
- Can make a datatype representing these trees

Parse Trees

- Notice that the different replacements didn't affect each other
  - Can effectively replace them in parallel
  - This is what "context free" means
- Tree structure
  - Non-terminal is a node
  - Terminal is a leaf
  - Edge is application of rule from grammar
- Can make a datatype representing these trees

Parse Trees

- Notice that the different replacements didn't affect each other
  - Can effectively replace them in parallel
  - This is what "context free" means
- Tree structure
  - Non-terminal is a node
  - Terminal is a leaf
  - Edge is application of rule from grammar
- Can make a datatype representing these trees

```
(define-type Expr
  (NumLit [n : Number])
  (Plus [left : Expr]
        [right : Expr])
  (Times [left : Expr]
         [right : Expr]))
```

Abstract Syntax

Abstract Syntax

```
(define-type Expr
  (NumLit [n : Number])
  (Plus [left : Expr]
        [right : Expr])
  (Times [left : Expr]
         [right : Expr]))
```

- This is called the *abstract syntax* for the programming language

Abstract Syntax

```
(define-type Expr
  (NumLit [n : Number])
  (Plus [left : Expr]
        [right : Expr])
  (Times [left : Expr]
         [right : Expr]))
```

- This is called the *abstract syntax* for the programming language
- A value of this type is called an *abstract syntax tree*

Abstract Syntax

```
(define-type Expr
  (NumLit [n : Number])
  (Plus [left : Expr]
        [right : Expr])
  (Times [left : Expr]
         [right : Expr]))
```

- This is called the *abstract syntax* for the programming language
- A value of this type is called an *abstract syntax tree*
  - AST for short

Parsing

- The process of turning source code (linear string) into abstract syntax (tree)

Parsing

- The process of turning source code (linear string) into abstract syntax (tree)
  - Turns the program into a thing we process recursively

Parsing

- The process of turning source code (linear string) into abstract syntax (tree)
  - Turns the program into a thing we process recursively
  - Tree structure mirrors structure of the program

Parsing

- The process of turning source code (linear string) into abstract syntax (tree)
  - Turns the program into a thing we process recursively
  - Tree structure mirrors structure of the program
- Can fail

Parsing

- The process of turning source code (linear string) into abstract syntax (tree)
  - Turns the program into a thing we process recursively
  - Tree structure mirrors structure of the program
- Can fail
  - What if the string isn’t generated by the grammar?

Parsing in this class

- Parsing is an interesting problem

Parsing in this class

- Parsing is an interesting problem
- But it's not an interesting *programming languages* problem

Parsing in this class

- Parsing is an interesting problem
- But it's not an interesting *programming languages* problem
- We will use Racket/Flit features to do most of the parsing for us

Parsing in this class

- Parsing is an interesting problem
- But it's not an interesting *programming languages* problem
- We will use Racket/Flit features to do most of the parsing for us
    - Use quoting to write s-expressions directly

Parsing in this class

- Parsing is an interesting problem
- But it's not an interesting *programming languages* problem
- We will use Racket/Flit features to do most of the parsing for us
    - Use quoting to write s-expressions directly
        - Does the hard work of figuring out nested brackets

The S-Expression Type

- Symbolic expressions

The S-Expression Type

- Symbolic expressions
  - Goes back to John McCarthy, LISP, early days of AI at MIT

The S-Expression Type

- Symbolic expressions
  - Goes back to John McCarthy, LISP, early days of AI at MIT
  - s-exp for short

The S-Expression Type

- Symbolic expressions
  - Goes back to John McCarthy, LISP, early days of AI at MIT
  - s-exp for short
- In Flit, `'(foo bar ...)` always has type `S-Exp`

The S-Expression Type

- Symbolic expressions
  - Goes back to John McCarthy, LISP, early days of AI at MIT
  - s-exp for short
- In Flit, `'(foo bar ...)` always has type `S-Exp`
- We can imagine it as being something like:

The S-Expression Type

- Symbolic expressions
  - Goes back to John McCarthy, LISP, early days of AI at MIT
  - s-exp for short
- In Flit, `'(foo bar ...)` always has type `S-Exp`
- We can imagine it as being something like:

The S-Expression Type

- Symbolic expressions
  - Goes back to John McCarthy, LISP, early days of AI at MIT
  - s-exp for short
- In Flit, `'(foo bar ...)` always has type `S-Exp`
- We can imagine it as being something like:

```
(define-type S-Expr
  (Identifier [x : Symbol])
  (NumLit [n : Number]
  (BoolLit [b : Boolean]))
  ...
  (List [s-exps : (Listof S-Expr)]))
```

The Symbol Type

- A useful built-in Flit type for names of things

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

```
'x
'hello
'foobar
'cheese
```

- Only relevant operation is:

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

```
'x
'hello
'foobar
'cheese
```

- Only relevant operation is:

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

```
'x
'hello
'foobar
'cheese
```

- Only relevant operation is:

```
symbol=? : (Symbol Symbol -> Boolean)
```

  - Checks if two symbols are equal

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

```
'x
'hello
'foobar
'cheese
```

- Only relevant operation is:

```
symbol=? : (Symbol Symbol -> Boolean)
```

  - Checks if two symbols are equal
- Kind of like strings, but there's no concept of "substring"

The Symbol Type

- A useful built-in Flit type for names of things
  - e.g. variables in a programming language we're interpreting
- Symbol literals:

```
'x
'hello
'foobar
'cheese
```

- Only relevant operation is:

```
symbol=? : (Symbol Symbol -> Boolean)
```

  - Checks if two symbols are equal
- Kind of like strings, but there's no concept of "substring"
  - Just a tag that we can create and check if two are the same

S-Expression in Flit

- The backtick ‘ in Racket says “interpret the next thing as an s-exp”

S-Expression in Flit

- The backtick ‘ in Racket says “interpret the next thing as an s-exp”

S-Expression in Flit

- The backtick ‘ in Racket says “interpret the next thing as an s-exp”

```
`a
`(+ 2 3)
`(a b c d (+ 2 3) #t (#f #f))
```

- We’ll use backtick as the first half of our parser

S-Expression in Flit

- The backtick ‘ in Racket says “interpret the next thing as an s-exp”

```
`a
`(+ 2 3)
`(a b c d (+ 2 3) #t (#f #f))
```

- We’ll use backtick as the first half of our parser
  - Easier to deal with s-expressions than strings

S-Expression Operations

- Many useful built-in functions

S-Expression Operations

- Many useful built-in functions

S-Expression Operations

- Many useful built-in functions

```
(s-exp->list `(a b (+ 1 2) #f))
(s-exp-number? `x )
(s-exp-number? `3 )
(s-exp-symbol? `x )
(s-exp-symbol?  `(+ 2 3))
(symbol=? '+
          (s-exp->symbol (first (s-exp->list `(+ 2 3)))))
```

S-Expression Operations

- Many useful built-in functions

```
(s-exp->list `(a b (+ 1 2) #f))
(s-exp-number? `x )
(s-exp-number? `3 )
(s-exp-symbol? `x )
(s-exp-symbol?  `(+ 2 3))
(symbol=? '+
          (s-exp->symbol (first (s-exp->list `(+ 2 3)))))
```

```
(list `a `b `(+ 1 2) `#f)
#f
#t
#t
#f
#t
```

From S-exp to AST

- S-expression is a tree

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it’s unsupported

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it’s unsupported
- If it’s a list

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it’s unsupported
- If it’s a list
  - Check the first thing in the list

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it’s unsupported
- If it’s a list
  - Check the first thing in the list
  - If it’s a keyword, check that we have the right number of arguments

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it’s unsupported
- If it’s a list
  - Check the first thing in the list
  - If it’s a keyword, check that we have the right number of arguments
  - If we do, (try to) parse each argument

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it’s unsupported
- If it’s a list
  - Check the first thing in the list
  - If it’s a keyword, check that we have the right number of arguments
  - If we do, (try to) parse each argument
- Otherwise, fail

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it's unsupported
- If it's a list
  - Check the first thing in the list
  - If it's a keyword, check that we have the right number of arguments
  - If we do, (try to) parse each argument
- Otherwise, fail
- Uses the `s-exp-match?` function

From S-exp to AST

- S-expression is a tree
  - Might not be a tree representing anything in our language
- If s-exp is literal
  - e.g. a number
  - Generate the literal in our AST
  - Error if it's unsupported
- If it's a list
  - Check the first thing in the list
  - If it's a keyword, check that we have the right number of arguments
  - If we do, (try to) parse each argument
- Otherwise, fail
- Uses the `s-exp-match?` function
  - Don't need to memorize how it works, we'll give you the parsers in this class

Example Parser

Example Parser

```
(define (parse [s : S-Exp]) : Expr
  (cond
    [(s-exp-match? `NUMBER s) (NumLit (s-exp->number s))]
    [(s-exp-match? `{+ ANY ANY} s)
     (Plus (parse (second (s-exp->list s)))
           (parse (third (s-exp->list s))))]
    [(s-exp-match? `{* ANY ANY} s)
     (Times (parse (second (s-exp->list s)))
            (parse (third (s-exp->list s))))]
    [else (error 'parse "invalid input")]))
```